

Arbitrary file read through path traversal in run-llama/llama_index

✓ Valid

Reported on Apr 1st 2025

Description:

The code in `generic_utils.py` has a path traversal vulnerability, which allows an attacker to control the file path provided to the `ImageDocument` class. This can lead to the reading of arbitrary files on the server, including sensitive system files, through base64 encoding and decoding of files. This vulnerability occurs due to improper validation or sanitization of the `image_path` input when it is provided to the `encode_image` function.

Root Cause:

The root cause of the vulnerability lies in the `encode_image` function within the `generic_utils.py` file. The function opens the file specified by `image_path` directly, without any validation or sanitization of the path. This enables an attacker to manipulate the path by using path traversal sequences (e.g., `../../../../`) to access files outside the intended directory. As a result, an attacker could potentially access sensitive files on the server, such as `/etc/passwd`, which contains information about system users.

The following function in `generic_utils.py` is vulnerable:

```
with open(image_path, "rb") as image_file:
    return base64.b64encode(image_file.read()).decode("utf-8")
```

Steps to Reproduce:

- Deploy the vulnerable version of the application containing the code in `generic_utils.py`.
- Start a Flask server with the provided Proof of Concept (PoC) code, which exposes an API endpoint (`/upload`) that accepts an image path via a `POST` request.
- Craft a `POST` request containing a malicious file path in the `image_path` parameter. For example:

```
curl -X POST -F "image_path=../../../../../../etc/passwd" http://localhost:5000/upload
```

- If the server is vulnerable, it will attempt to base64 encode the contents of the file at the given path, which in this case could be `/etc/passwd`, and return the base64-encoded file content in the response.

PoC (Proof of Concept):

```
from flask import Flask, request
import base64
from llama_index.core.schema import ImageDocument
from llama_index.core.multi_modal_llms.generic_utils import image_documents_to_base64

app = Flask(__name__)

@app.route("/upload", methods=["POST"])
def upload_image():
    image_path = request.form.get("image_path") # Attacker-controlled input
    doc = ImageDocument(image_path=image_path)
    try:
        encoded = image_documents_to_base64([doc])
        if encoded:
            return {"encoded": encoded[0], "decoded": base64.b64decode(encoded[0])}.
            return {"error": "No content returned"}
    except Exception as e:
        return {"error": str(e)}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Attack Scenario:

- **Attacker's Role:** The attacker must send a malicious `POST` request to the vulnerable Flask endpoint (`/upload`) with a crafted `image_path` pointing to sensitive files on the server.
- **Malicious Input:** The attacker may specify paths like:
 - `../../../../etc/passwd`
 - `/etc/shadow`
 - `../../../../etc/hosts`
- **Exploitation:** The Flask application will attempt to base64-encode the contents of the file at the attacker-controlled path. If the path is valid, the attacker will receive the base64-encoded content of the file in the response. This could expose sensitive system files, potentially allowing attackers to leak or manipulate crucial data.

Suggested Fix:

To mitigate this vulnerability, the following steps should be taken:

- **Sanitize Input:** Always validate and sanitize the `image_path` to ensure it does not contain path traversal sequences (e.g., `../`) or absolute paths that could lead to file system traversal.

Example fix using `os.path.abspath()` :

```
import os

def sanitize_path(image_path):
    absolute_path = os.path.abspath(image_path)
    # Check if the path is inside a specific directory (e.g., /images)
    if not absolute_path.startswith("/path/to/images"):
        raise ValueError("Invalid file path")
    return absolute_path
```

- **Restrict File Access:** Ensure that the file path is only allowed to reference files within a predefined, restricted directory, such as a `media` or `uploads` folder.

Impact

The impact of this vulnerability is critical due to the following reasons:

- **Information Disclosure:** An attacker could gain access to sensitive system files such as `/etc/passwd` (which contains user information) or `/etc/shadow` (which contains hashed passwords).
- **Privilege Escalation:** If the server is misconfigured or running with elevated privileges, attackers might be able to access additional system files or even gain control over the server.
- **Data Integrity:** If attackers can access sensitive files, they might be able to tamper with them, leading to data corruption or manipulation.

⚙️ We are processing your report and will contact the `run-llama/llama_index` team within 24 hours. 10 months ago

🔗 A [GitHub Issue](#) asking the maintainers to create a `SECURITY.md` exists 10 months ago



Manan Patel commented 9 months ago

Researcher

Any updates?>



Manan Patel commented 9 months ago

Researcher

Are maintainers active for this project?

⚙️ This report has now been passed to internal triage and should be resolved within 14 days. 9 months ago



Manan Patel commented 8 months ago

Researcher

Hello @Huntr Team!! It's 2 weeks but no one from internal triage team have looked at this. Can you please check?



Clelia (Astra) Bertelli commented 8 months ago

Maintainer

Heya

I'm not sure how this really works, but the vulnerability is there so thank you for reporting it!

We have a [PR](#) that is trying to fix this issue by checking that the files we provide are actually images, and this indeed prevents the PoC you provided from returning the encoded/decoded version of '/etc/passwd' or other non-image files that might be dangerous to expose



A **run-llama/llama_index** maintainer has acknowledged this report 8 months ago



The scheduled publication date was automatically extended from 30th Jun 2025 to 7th Jul 2025 due to the maintainers acknowledgement of the report 8 months ago



Manan Patel commented 8 months ago

Researcher

I see that the fix has already been pushed. Here's the relevant commit for reference: [cdeaab9](#)

Could you please let me know the expected timeline for the CVE assignment and bounty?



Logan validated this vulnerability 7 months ago

\$ **Oxmanan** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs



The researcher's credibility has increased: +7



CVE-2025-6209 assigned to this report. 7 months ago



Logan marked this as fixed in **0.12.41** with commit **cdeaab** 7 months ago

\$ **Clelia (Astra) Bertelli** has been awarded the fix bounty ✓



We have notified the **run-llama/llama_index** maintainers about this report in their weekly follow-up 7 months ago



We have sent a warning to the **run-llama/llama_index** team to inform them that this report will be published in 48 hours 7 months ago



This vulnerability has now been published 7 months ago



[Sign in](#) to join this conversation

CVE
CVE-2025-6209
Published

Vulnerability Type
CWE-29: Path Traversal: '..\filename'

Severity

High (7.5)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
0.12.27

Visibility
Public

Status
Fixed

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



Manan Patel
@0xmanan
MIDDLEWEIGHT



Fixed by



Clelia (Astra) Bertelli
@astrabert
UNPROVEN



